Pandas Time Series

Today we are going to look at CO_2 data and try to learn about time series. I will introduce it and then you will be in charge of analyzing CO_2 data from Mauna Lao. We are going to start by looking at the data from the Scripps Pier.

In [1]:
```
%matplotlib ipympl
import pandas as pd
import numpy as np
import matplotlib.pylab as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages
np.set_printoptions(legacy='1.25') #removes the funny printing
```

First go get CO2 data for Scripps. http://scrippsco2.ucsd.edu/data/atmospheric_co2/ljo and the La Jolla Pier.

- I did flask daily values.
- Download and Open the CSV.
- rename the columns and called them Date, HR, Excel, Year, Flask, Flag, CO2. (see picture below)
- I would then save as excel.
- Add the date you downloaded the file to the name so you know it
- daily_merge_co2_ljo_25Oct2025.xlsx
- Then we can skip the rows and columns we don't want. You might have to change this. The number wiggles

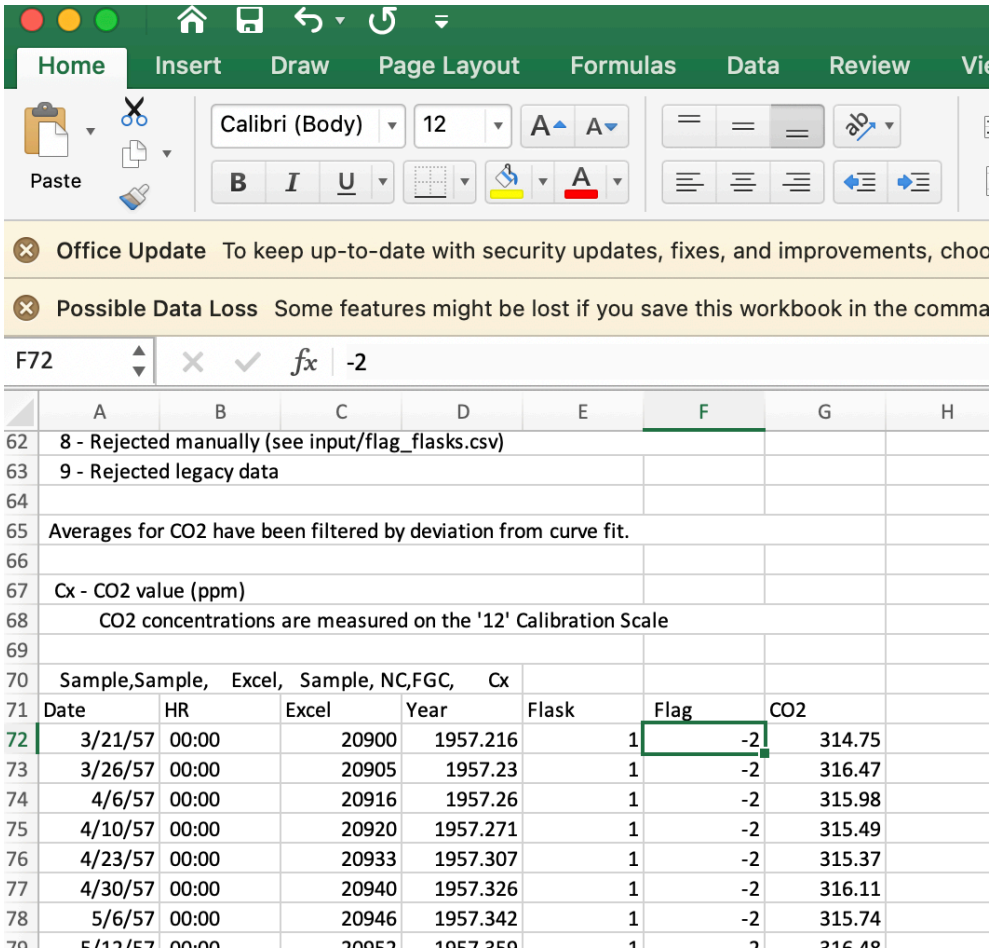
I kept all the information in place and used skiprows to skip what we don't want.
http://pandas.pydata.org/pandas-docs/stable/

Start by just reading in the data. do you get the data? Then get more clever as you go. Here is the read_csv information. https://pandas.pydata.org/pandas-docs/stable/generated/pandas.read_csv.html or http://pandas.pydata.org/pandas-docs/stable/io.html

You just need to try and do to learn.

In [2]:
```
from IPython.display import Image
Image('excel_header.png',width=600)
```

Out[2]:



In [5]:
```
dfS=pd.read_excel('daily_merge_co2_ljo_25Oct2025.xlsx',skiprows=70)
```

I always look at the dataframe, the "info" and the "columns" when I begin with a new dataset

In [10]:
```
dfS
```

Out[10]:

	Date	HR	Excel	Year	Flask	Flag	CO2
0	1957-03-21	00:00	20900.00	1957.216	1	-2	314.75
1	1957-03-26	00:00	20905.00	1957.230	1	-2	316.47
2	1957-04-06	00:00	20916.00	1957.260	1	-2	315.98
3	1957-04-10	00:00	20920.00	1957.271	1	-2	315.49
4	1957-04-23	00:00	20933.00	1957.307	1	-2	315.37
...	...	...	...	...	...	...	...
1546	2024-01-23	16:03	45314.67	2024.062	8	0	427.14
1547	2024-02-02	14:45	45324.61	2024.089	8	0	426.51
1548	2024-02-29	15:05	45351.63	2024.163	8	1	-74257.39
1549	2024-03-12	13:27	45363.56	2024.196	8	0	427.74
1550	2024-04-04	14:22	45386.60	2024.258	8	0	426.97

1551 rows × 7 columns

In [9]:
```
dfS.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1551 entries, 0 to 1550
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1551 non-null   datetime64[ns]
 1   HR      1551 non-null   object
 2   Excel   1551 non-null   float64
 3   Year    1551 non-null   float64
 4   Flask   1551 non-null   int64
 5   Flag    1551 non-null   int64
 6   CO2     1551 non-null   float64
dtypes: datetime64[ns](1), float64(3), int64(2), object(1)
memory usage: 84.9+ KB
```

In [13]:
```
dfS.columns
```

Out[13]:
```
Index(['Date', 'HR', 'Excel', 'Year', 'Flask', 'Flag', 'CO2'], dtype='object')
```

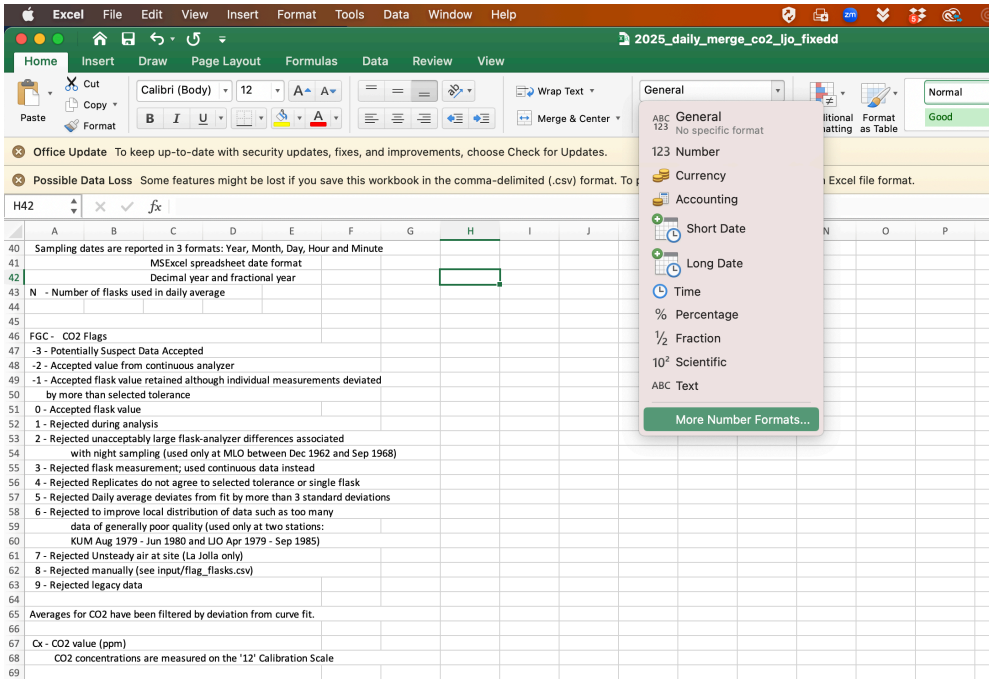
Big Date Time Issue!!!!!!

Some days Python converts date times nicelys and sometimes poorly. I find if you do 4 digit years it goes much smoother and consistently works. You can specify date formats for reading in but I wouuld just try and do 4 digit years and change in excel. These are two good options: mm/dd/yyyy or yyyy-mm-dd Instructions are below. This was the y2k bug!

- Change to 4 digit years in excel.
- The pictures don't show it but highlight column A then follow the instructions

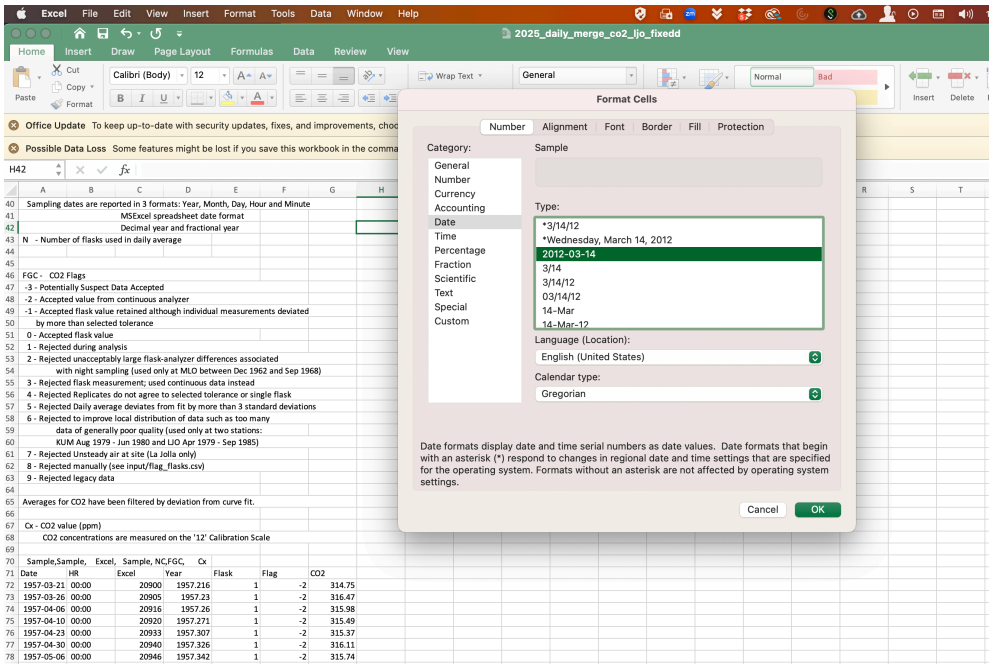
In [84]:
```
Image('excel-choose.png',width=600)
```

Out[84]:



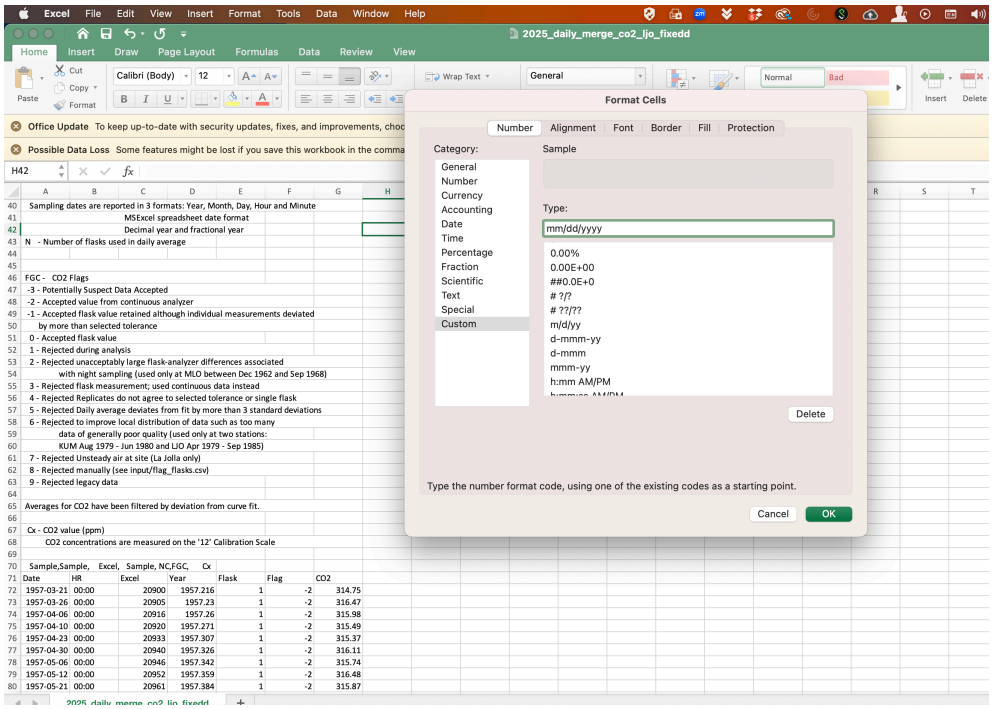
In [85]:
```
Image('excel-date.png',width=600)
```

Out[85]:



In [86]:
```
Image('excel-custom.png',width=600)
```

Out[86]:



Now read in the file again

In [15]:
```
dfS=pd.read_excel('daily_merge_co2_ljo_25Oct2025.xlsx',skiprows=70)
```

In [16]:
```
dfS.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1551 entries, 0 to 1550
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1551 non-null   datetime64[ns]
 1   HR      1551 non-null   object
 2   Excel   1551 non-null   float64
 3   Year    1551 non-null   float64
 4   Flask   1551 non-null   int64
 5   Flag    1551 non-null   int64
 6   CO2     1551 non-null   float64
dtypes: datetime64[ns](1), float64(3), int64(2), object(1)
memory usage: 84.9+ KB
```

datetime!

See how it says datetime64[ns] next to Date. That is critical. It means python knows it is a date and we can do date stuff!

But we only want to the Date, Hr, Flags, and CO2. So just grab those columns using use_cols.

We can give column numbers or column names

also set filename so you only type it once and makes it look clearner

In [18]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70,usecols=[0,1,5,6])
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1551 entries, 0 to 1550
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1551 non-null   datetime64[ns]
 1   HR      1551 non-null   object
 2   Flag    1551 non-null   int64
 3   CO2     1551 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 48.6+ KB
None
```

In [20]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70
                  ,usecols=['Date','HR','Flag','CO2'])
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1551 entries, 0 to 1550
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1551 non-null   datetime64[ns]
 1   HR      1551 non-null   object
 2   Flag    1551 non-null   int64
 3   CO2     1551 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 48.6+ KB
None
```

We want a datetime index!

If we can sett the datetime to an index it gives us more power.

The date time index is critical and can really mess you up if you don't have it!

Don't forget this!!! Always check for it!

In [22]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70
                        ,usecols=['Date','HR','Flag','CO2'],index_col='Date')
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1551 entries, 1957-03-21 to 2024-04-04
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   HR      1551 non-null   object 
 1   Flag    1551 non-null   int64  
 2   CO2     1551 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 48.5+ KB
None
```

See how it says

DatetimeIndex: 1551 entries, 1957-03-21 to 2024-04-04 That is perfect!

Three Side notes for your reference

look at these and copy and paste to run them but don't get caught up here. Just for refernce.

Side note one. You can combine Date and time.

But back to reading in. We can also add the hours to the date and set the index and dateime all at once. BUT YOU NEED the double brackets on parse dates for this to work because it is a list you are using

In [24]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70
                    ,usecols=['Date','HR','Flag','CO2'],parse_dates=[['Date','HR']]
                    ,index_col='Date_HR')
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1551 entries, 1957-03-21 00:00:00 to 2024-04-04 14:22:00
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Flag    1551 non-null   int64  
 1   CO2     1551 non-null   float64
dtypes: float64(1), int64(1)
memory usage: 36.4 KB
None
/var/folders/dw/0b9k7c_s7nj6fspmx6n2gbk40000gp/T/ipykernel_91692/20347838.py:2: UserWarning: Coul
d not infer format, so each element will be parsed individually, falling back to `dateutil`. To e
nsure parsing is consistent and as-expected, please specify a format.
  dfS=pd.read_excel(filename,skiprows=70
```

Let's fix the warning.

You just need to specifiy the date_format.
This site describes the different date formats. https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior

In [25]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70
                  ,usecols=['Date','HR','Flag','CO2']
                  ,parse_dates=[['Date','HR']]
                  ,date_format='%m/%d/%Y'
                  ,index_col='Date_HR')
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1551 entries, 1957-03-21 00:00:00  00:00 to 2024-04-04 00:00:00  14:22
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Flag    1551 non-null   int64
 1   CO2     1551 non-null   float64
dtypes: float64(1), int64(1)
memory usage: 36.4+ KB
None
```

In [ ]:

In [ ]:

Side note two

Just for reference

This is a fun trick. We can use python to grab the csv file for us off of the website! No need to download. But something is wrong with the file and we will lose the first row of data. I needed to adjust skip rows and then you can see the problem with the names. It changes each year. But this is great because you always get the most up to date file. Do not type the url. Go right click on the file and say copy link and paste in.

You can see the files on this website. http://scrippsco2.ucsd.edu/data/atmospheric_co2/ljo

In [28]:
```
url='https://scrippsco2.ucsd.edu/assets/data/atmospheric/stations/merged_in_situ_and_flask/daily
dfS=pd.read_csv(url,skiprows=71,usecols=[0,1,5,6],names=['Date','HR','Flag','CO2']
                      ,parse_dates=[['Date','HR']]
                      ,index_col='Date_HR',header=None)
print (dfS.info())
```

```
/var/folders/dw/0b9k7c_s7nj6fspmx6n2gbk40000gp/T/ipykernel_91692/2515497085.py:2: FutureWarning:
Support for nested sequences for 'parse_dates' in pd.read_csv is deprecated. Combine the desired
columns with pd.to_datetime after parsing instead.
  dfS=pd.read_csv(url,skiprows=71,usecols=[0,1,5,6],names=['Date','HR','Flag','CO2']
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1551 entries, 1957-03-21 00:00:00 to 2024-04-04 14:22:00
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Flag    1551 non-null   int64
 1   CO2     1551 non-null   float64
dtypes: float64(1), int64(1)
memory usage: 36.4 KB
None
```

Side note three

- You can read in the data then seperately set the datetime and index.
- This will get rid of the warning.
- This is nice to be able to read online data

In [46]:
```
url='https://scrippsco2.ucsd.edu/assets/data/atmospheric/stations/merged_in_situ_and_flask/daily
dfS=pd.read_csv(url,skiprows=71,usecols=[0,1,5,6],names=['Date','HR','Flag','CO2'],header=None)
```

```
dfS['Date_HR']=dfS.Date.astype(str) + ' ' + dfS.HR.astype(str)
dfS['Date_HR']=pd.to_datetime(dfS['Date_HR'], format="%Y-%m-%d %H:%M")
dfS.set_index('Date_HR',inplace=True)
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1551 entries, 1957-03-21 00:00:00 to 2024-04-04 14:22:00
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Date    1551 non-null   object 
 1   HR      1551 non-null   object 
 2   Flag    1551 non-null   int64  
 3   CO2     1551 non-null   float64
dtypes: float64(1), int64(1), object(2)
memory usage: 60.6+ KB
None
```

Now lets start with the data.

keep it simple. Use your nice excel data

I always

- set my filename
- read it in and set the index
- do .info()
- make sure the index is a datetimeindex
- print the columns so you have the names
- just look at the dataframe to make sure things look ok
- Then you are ready for plots and analyses

In [47]:
```
filename='daily_merge_co2_ljo_25Oct2025.xlsx'
dfS=pd.read_excel(filename,skiprows=70
                  ,usecols=['Date','HR','Flag','CO2'],parse_dates=[['Date','HR']]
                  ,index_col='Date_HR')
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1551 entries, 1957-03-21 00:00:00 to 2024-04-04 14:22:00
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Flag    1551 non-null   int64  
 1   CO2     1551 non-null   float64
dtypes: float64(1), int64(1)
memory usage: 36.4 KB
None
/var/folders/dw/0b9k7c_s7nj6fspmx6n2gbk40000gp/T/ipykernel_91692/20347838.py:2: UserWarning: Coul
d not infer format, so each element will be parsed individually, falling back to `dateutil`. To e
nsure parsing is consistent and as-expected, please specify a format.
  dfS=pd.read_excel(filename,skiprows=70
```

In [48]:
```
dfS.columns
```

Out[48]:
```
Index(['Flag', 'CO2'], dtype='object')
```

In [49]:
```
dfS
```

Out[49]:

Date_HR	Flag	CO2
1957-03-21 00:00:00	-2	314.75
1957-03-26 00:00:00	-2	316.47
1957-04-06 00:00:00	-2	315.98
1957-04-10 00:00:00	-2	315.49
1957-04-23 00:00:00	-2	315.37
...	...	...
2024-01-23 16:03:00	0	427.14
2024-02-02 14:45:00	0	426.51
2024-02-29 15:05:00	1	-74257.39
2024-03-12 13:27:00	0	427.74
2024-04-04 14:22:00	0	426.97

1551 rows × 2 columns

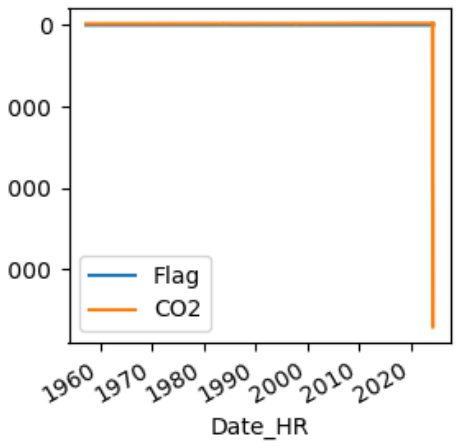
Now we can start thinking about the data. Data setup is critical! That is why we repeated doing it so many times. But now lets look at the data!

You are now an expert at getting data in. You can figure it out! Lets plot the date. We can do the pandas quick plotting.

In [59]:
```
fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
dfS.plot(ax=ax)
```

Out[59]:
```
<Axes: xlabel='Date_HR'>
```

Figure

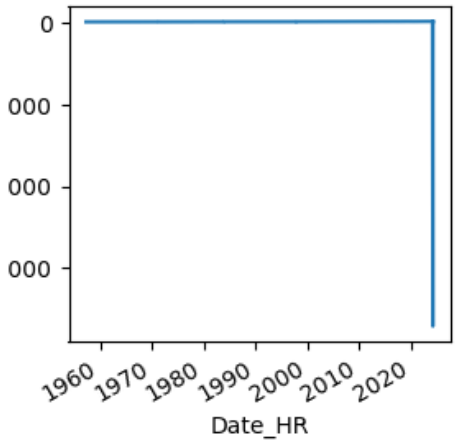


That looks horrible. Lets just plot the CO2 data

In [60]:
```
fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
dfS.CO2.plot(ax=ax)
```

Out[60]: <Axes: xlabel='Date_HR'>

Figure



There is obviosly some bad data points. Go look at the csv file header and figure out which values are good and only keep the good data.

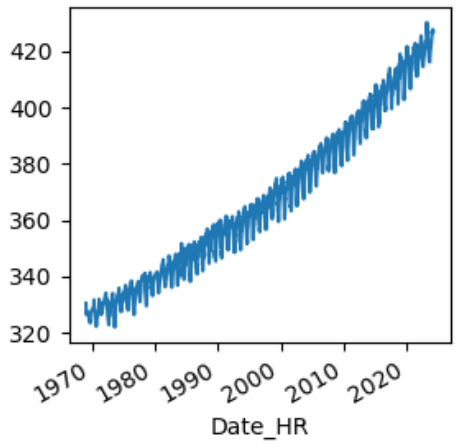
- Good data has a flag of 0.
- I am going to drop all the other data.

I just figured out this next step. Since we are filtering the data by flag we need to add the .copy(). This saves us a warning later on about SettingWithCopyWarning.

In [63]:
```
dfS=dfS[dfS.Flag==0].copy()   #this .copy is critical for stopping an error later.

fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
dfS.CO2.plot(ax=ax)
```

Out[63]: <Axes: xlabel='Date_HR'>

Figure



Now that is much nicer!

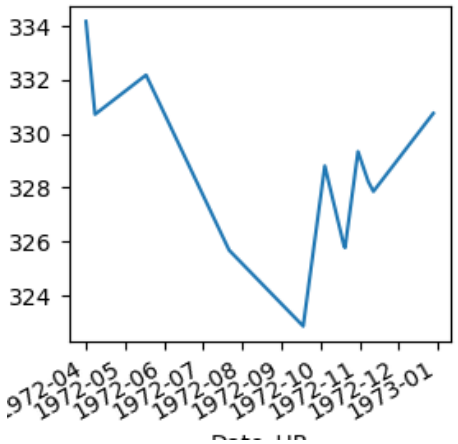
Now can you plot just the data from your birth year?

In [68]:
```
fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
```

```
dfS.loc['1972'].CO2.plot(ax=ax)
```

Out[68]: <Axes: xlabel='Date_HR'>

Figure



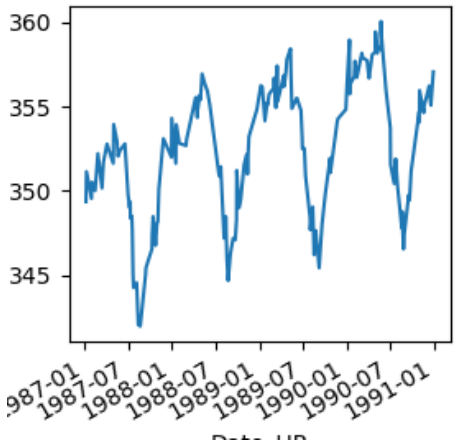
Remember all your slicing? We can also now slice by Date!!!

Now just plot it for the years you were in high school

In [65]:
```
fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
dfS.loc['1987':'1990'].CO2.plot(ax=ax)
```

Out[65]: <Axes: xlabel='Date_HR'>

Figure

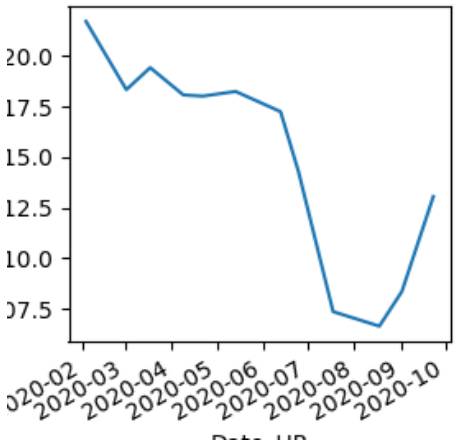


You can also go between dates

In [66]:
```
fig,ax=plt.subplots()
fig.set_size_inches(3,3) # I made this smaller to try and save room printing.  You can delete
dfS.loc['1/1/2020':'10/5/2020'].CO2.plot(ax=ax)
```

Out[66]: <Axes: xlabel='Date_HR'>

Figure



We are going to make a boxplot of CO2 by decade.

But first this is an important note.

- To make a new column you have to use the [] notation and not the . notation.
- See this example.

In [69]:
```
dfS['Year']=dfS.index.year  #This just gives you the year from the index
print (dfS.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1282 entries, 1969-01-29 15:47:00 to 2024-04-04 14:22:00
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Flag    1282 non-null   int64
 1   CO2     1282 non-null   float64
 2   Year    1282 non-null   int32
dtypes: float64(1), int32(1), int64(1)
memory usage: 67.3 KB
None
```

In [70]:
```
dfS
```

Out[70]:

Date_HR	Flag	CO2	Year
1969-01-29 15:47:00	0	330.50	1969
1969-02-18 16:28:00	0	326.50	1969
1969-02-21 14:12:00	0	326.58	1969
1969-04-01 15:00:00	0	327.62	1969
1969-06-18 15:11:00	0	326.00	1969
...	...	...	...
2023-12-21 15:23:00	0	423.48	2023
2024-01-23 16:03:00	0	427.14	2024
2024-02-02 14:45:00	0	426.51	2024
2024-03-12 13:27:00	0	427.74	2024
2024-04-04 14:22:00	0	426.97	2024

1282 rows × 3 columns

You can also make new columns by using math. What if you wanted to make a new column that was the CO2 times 10 minus 5? I would just make a new column first. If you need a new dataframe you could do that also.

In [71]:
```
# sto make a new column
dfS['CO2_10x-5']=dfS['CO2']*10.0-5.0
print (dfS.head())

#to make a new dataframe
df=pd.DataFrame()
df['CO2_10x-5']=dfS.CO2*10.0-5.0
print (df.head())
```

```
                     Flag     CO2  Year  CO2_10x-5
Date_HR
1969-01-29 15:47:00     0  330.50  1969     3300.0
1969-02-18 16:28:00     0  326.50  1969     3260.0
1969-02-21 14:12:00     0  326.58  1969     3260.8
1969-04-01 15:00:00     0  327.62  1969     3271.2
1969-06-18 15:11:00     0  326.00  1969     3255.0
                     CO2_10x-5
Date_HR
1969-01-29 15:47:00     3300.0
1969-02-18 16:28:00     3260.0
1969-02-21 14:12:00     3260.8
1969-04-01 15:00:00     3271.2
1969-06-18 15:11:00     3255.0
```

Flashback to earlier Pandas handouts

Now remember back to boxplots..... Can you make a boxplot of CO2 "by" decade? I couldn't figure it out how to do it without some googling.

Can you make sense of this? https://stackoverflow.com/questions/17764619/pandas-dataframe-group-year-index-by-decade

do you rememeber //?

In [72]:
```
print(1//10,2//20,5//10,10//10,12//10)
```

```
0 0 0 1 1
```

So now to get the decade you can

- //10
- multiply by 10

In [73]:
```
dfS.index.year//10*10
```

Out[73]:
```
Index([1960, 1960, 1960, 1960, 1960, 1960, 1960, 1970, 1970, 1970,
       ...
       2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020],
      dtype='int32', name='Date_HR', length=1282)
```

Now you can just make and set a new column

In [74]:
```
dfS['decade']=dfS.index.year//10*10
```

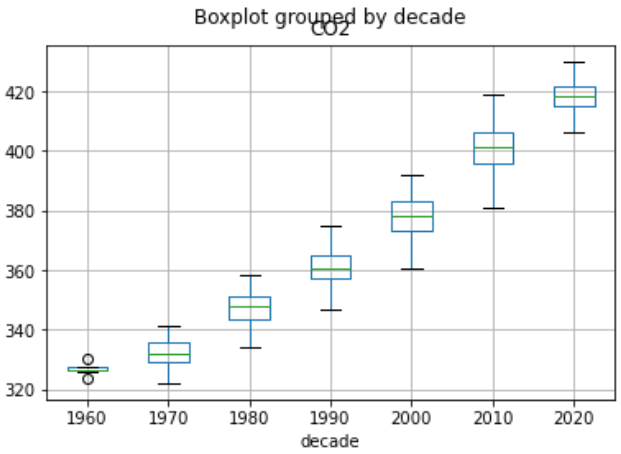
If you get a warning you can do this for SettingWithCopyWarning

In [75]:
```
dfS.loc[:,'decade']=dfS.index.year//10*10
```

Now make the boxplot!

In [48]:
```

```

Out[48]:
```
<AxesSubplot:title={'center':'CO2'}, xlabel='decade'>
```

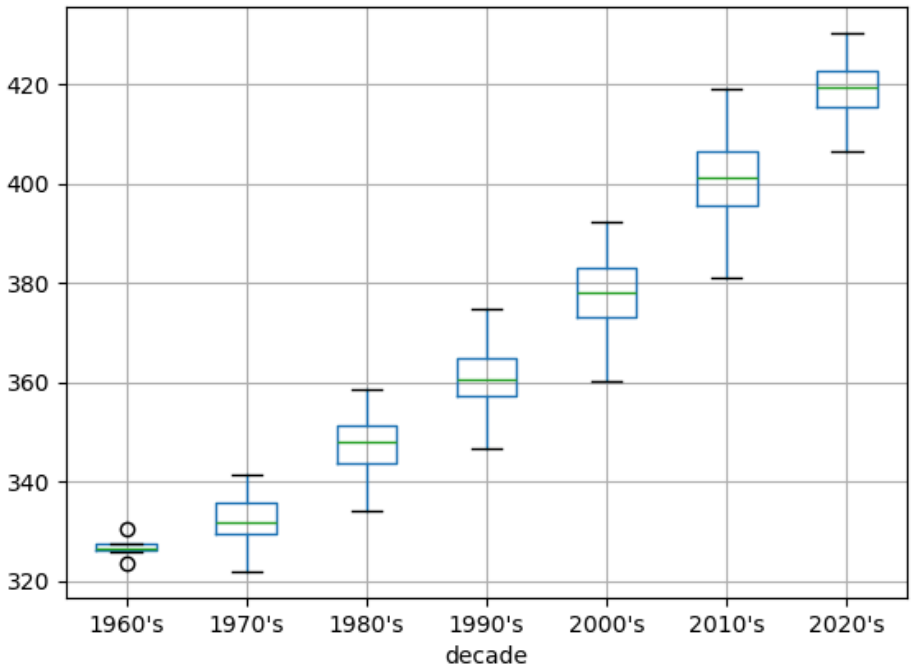


Making really pretty boxplots in Pandas isn't always perfect. Here are a few hints to use our fig and ax nomenclature. Fixing the ticks is harder. You have to add a new label for each tick and then blanks for the ones you don't want. just some tricks to put in your back pocket. Don't spend much time here. Just file it away for future reference,

In [81]:
```
fig,ax=plt.subplots()

dfS.boxplot(column='CO2',by='decade',ax=ax)
ax.set_xticklabels(["1960's","1970's","1980's","1990's","2000's","2010's","2020's"])
ax.set_title('') # you need to put a blank title to get rid of the default title
fig.suptitle('')#This has to go last and gets rid of some writing.
```

Out[81]:
```
Text(0.5, 0.98, '')
```

Figure

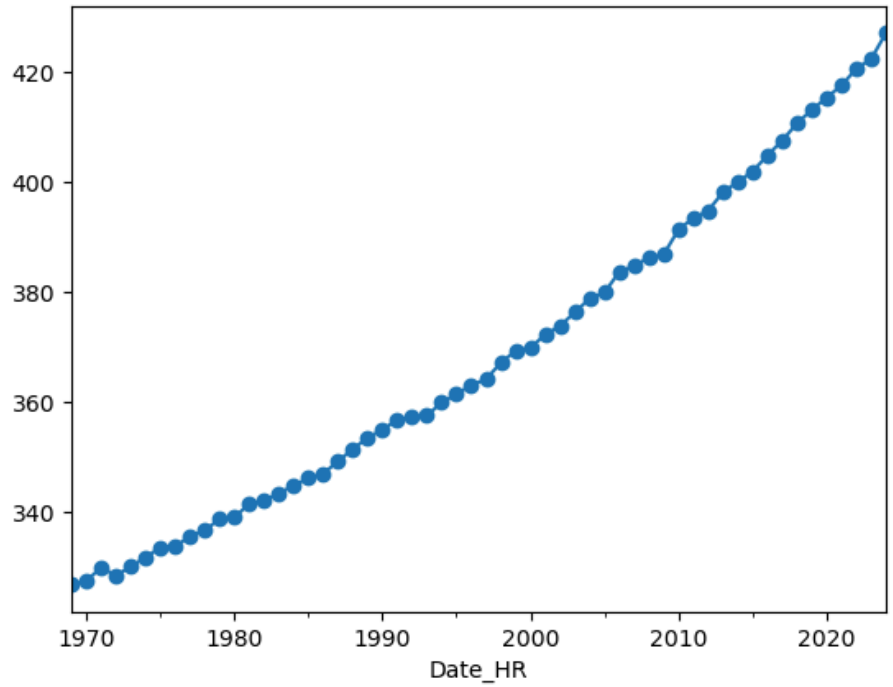


the next thing you might need to do is to resample by a larger time frame like a year. so what is the yearly mean? By resample we mean take all the data and find a subset of it. I am now just looking at the annual mean and plotting it all in one fell swoop. You need the np.mean b/c we are using an numpy function on our data. Here is a link to show you the options. https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases

In [84]:
```
fig,ax=plt.subplots()

dfS.CO2.resample('YE').mean().plot(ax=ax,marker='o') # For year you can try A, Y, or YE.  They a
```

Out[84]:
```
<Axes: xlabel='Date_HR'>
```

Figure

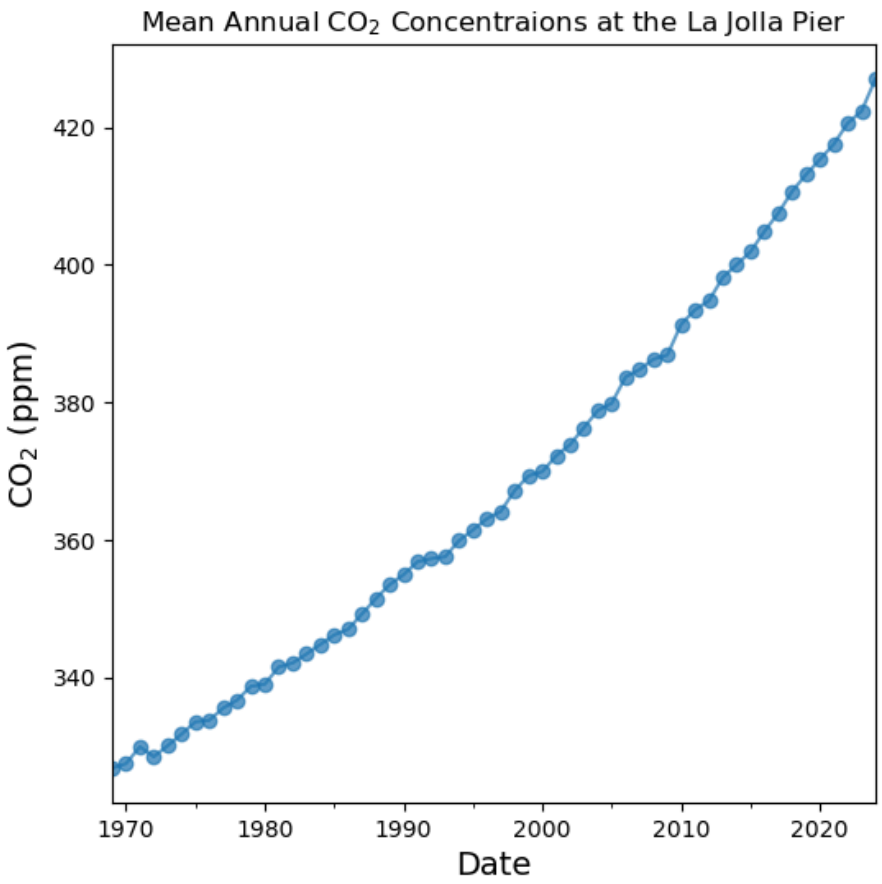


make it look nice a bit nicer

In [86]:
```
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
dfS.CO2.resample('YE').mean().plot(ax=ax,marker='o',alpha=0.7)
ax.set_ylabel('CO$_2$ (ppm)',fontsize=14)
ax.set_xlabel('Date',fontsize=14)
ax.set_title('Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Out[86]:
```
Text(0.5, 1.0, 'Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Figure

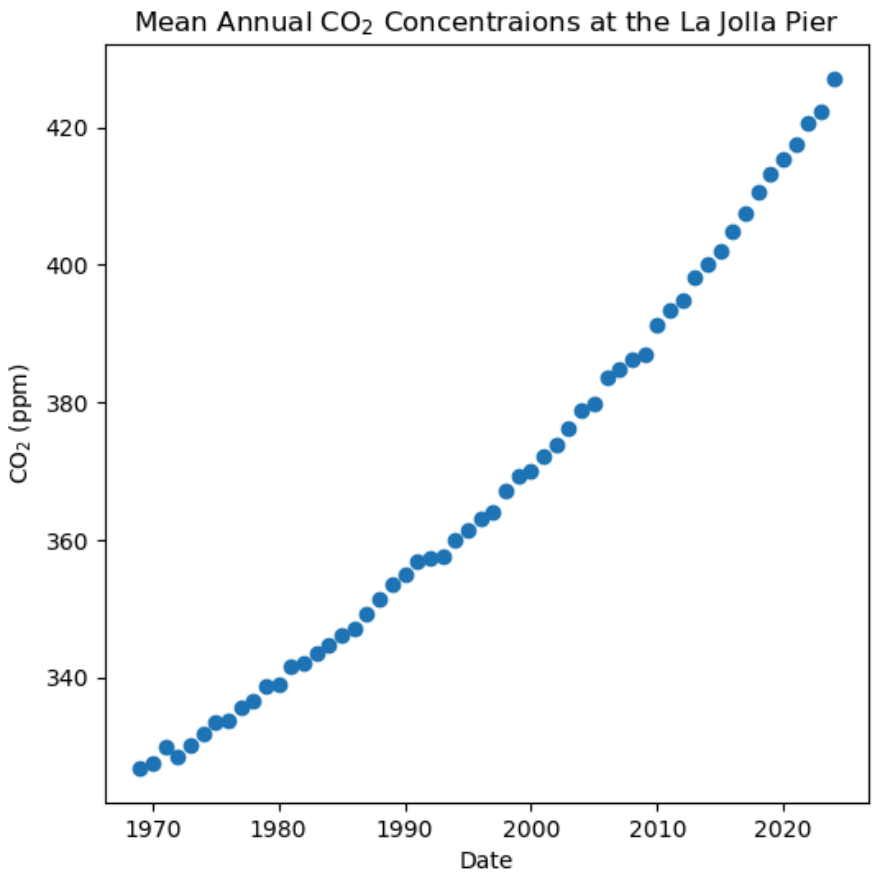


Here is how you would plot the data in scatter

In [87]:
```
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
x=dfS.CO2.resample('YE').mean().index.year
y=dfS.CO2.resample('YE').mean()
ax.scatter(x,y)
ax.set_ylabel('CO$_2$ (ppm)')
ax.set_xlabel('Date')
ax.set_title('Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Out[87]:
```
Text(0.5, 1.0, 'Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Figure

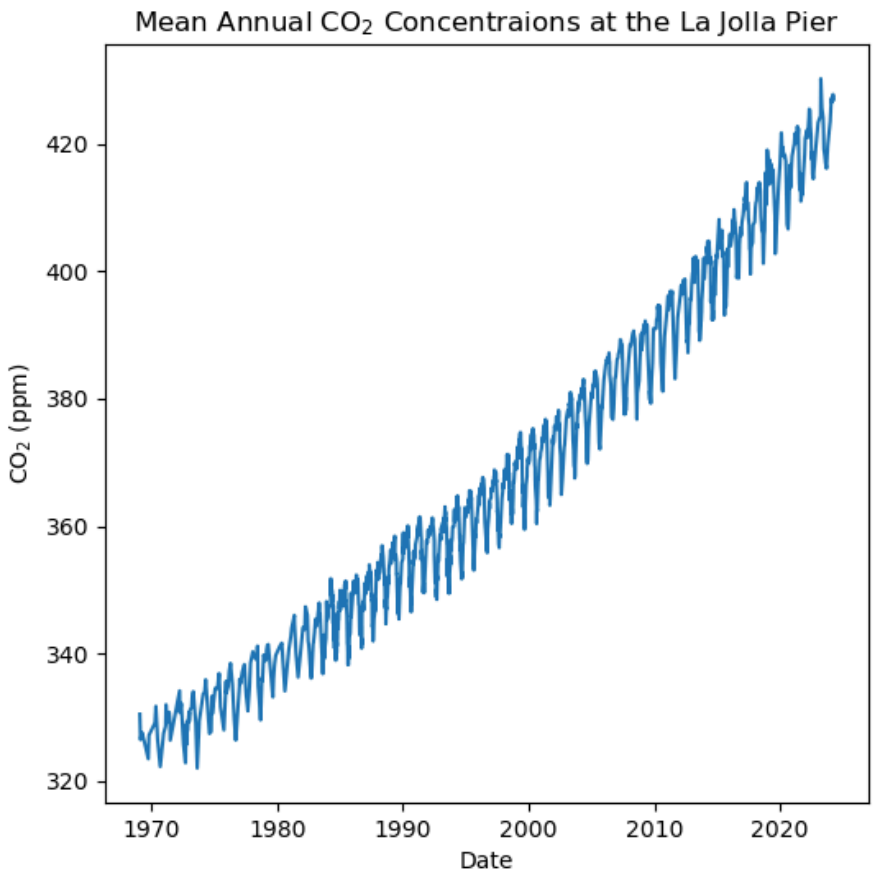


We could also plot all the data with plot outside of pandas.

In [88]:
```
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
x=dfS.CO2.index
y=dfS.CO2
ax.plot(x,y)
ax.set_ylabel('CO$_2$ (ppm)')
ax.set_xlabel('Date')
ax.set_title('Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Out[88]:
```
Text(0.5, 1.0, 'Mean Annual CO$_2$ Concentraions at the La Jolla Pier')
```

Figure



Now back to using Pandas. You can look online. There are so many ways to resample your data!

In [91]:
```
print( df_scripps.CO2.resample('QE').mean().head())  #THis just shows the quartely amounts!
# Just showing you there are more resample methods.  You can ignore
```

```
Date_HR
1969-03-31    327.860
1969-06-30    326.810
1969-09-30        NaN
1969-12-31    325.345
1970-03-31        NaN
Freq: QE-DEC, Name: CO2, dtype: float64
```

For the how part you can use any numpy array function. I have not found a list. So you can try many.

Linear Regression!

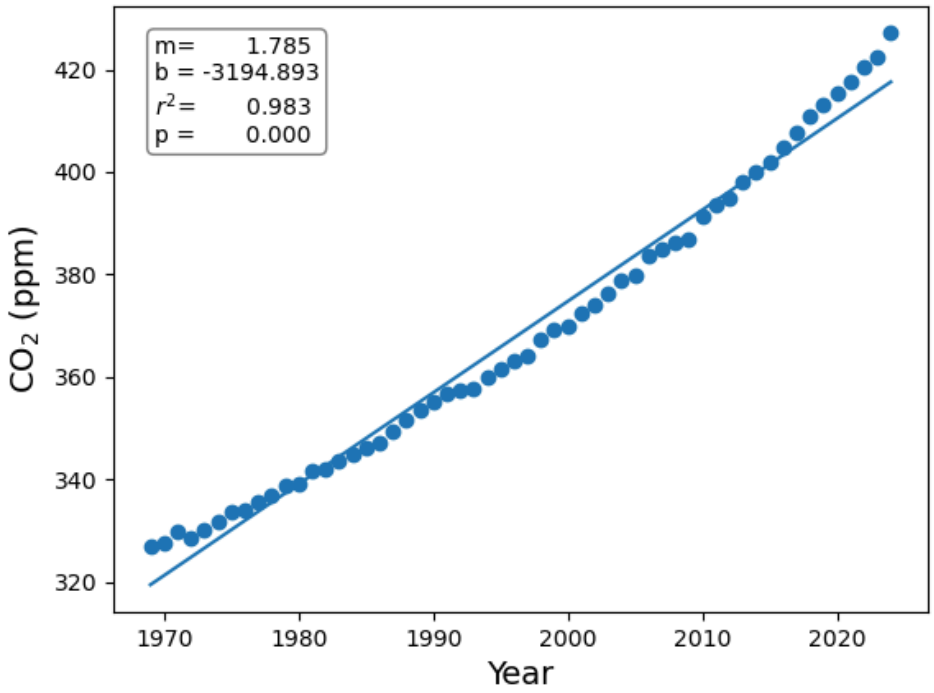
For now we will go back and use the annual mean values we did from resample and do linear regression

- Go back and set X and Y using resample for annual mean data.
- plot the x and y
- do linregress on the x and y
- make your graph look nice
- The intercept is non-sensical but do the other parameters make sense?

In [113…
```
# Your code here!
```

Out[113… `Text(0.5, 0, 'Year')`

Figure



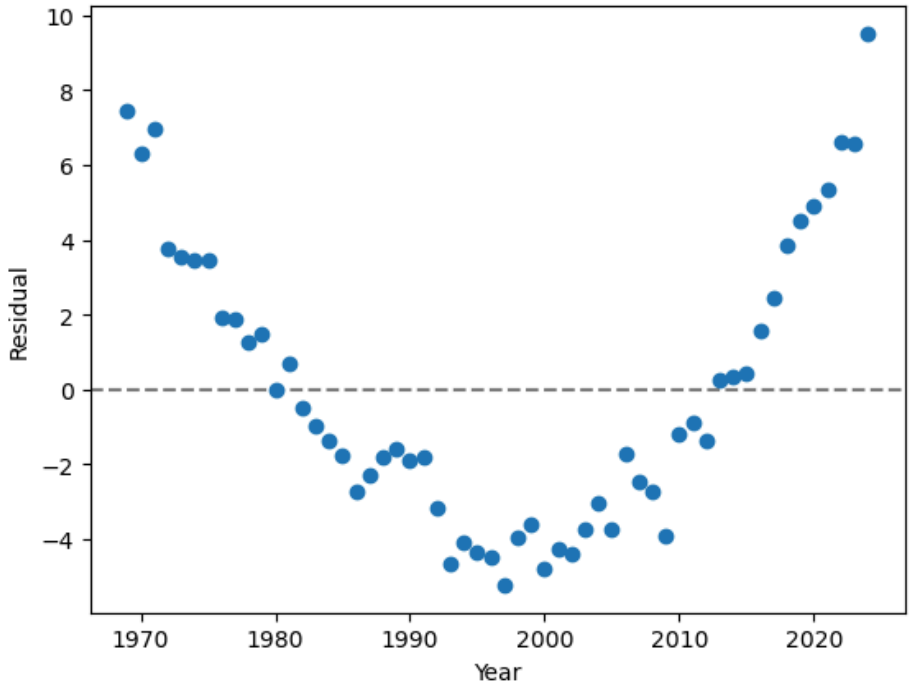
the r^2 is great but the curve looks non-linear. You can try to look at the graph "down the line" and you will see the line is straight but the points curve around it.

- So the error looks systematic and not random. could we find the residuals and plot them?
- So could we plot how far off the value is from the line for each year?
- So the residual is the difference in y-space for each value for each year. residuas=observed-predicted. I would make the residual by doing the math to look at the difference.

- Residual is Observed minus expected (observed-expected)
- expected=m*x+b from linregess
- so your resdiual beomces y-(mx+b)
- plot the residual versus x
- add the horizontal line by using ax.axhline(y=0)
- think about the data...

In [167…
```
# Your code here
fig,ax=plt.subplots()
```

Out[167… `<matplotlib.lines.Line2D at 0x142271640>`



So linear ain't working!

We can use poly1d to fit a polynominal!

So lets figure out how to use poly1d and see how a second order equation fits the data! poly1d has lots of cool tricks!

Here is the description https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.poly1d.html

Remember for a polynominal 1st order y=ax+b

2nd order $y=ax^2+bx+c$

3rd order $y=ax^3+bx^2+cx+d$

etc...

How it works. Like linregress. You pass your x,y and now the order to poly1d

set fit_order to 2 for a second order polynominal fit_order=2

Now use polyfit

a=np.polyfit(x,y,fit_order)

Now it gets cool.

a is now a python class. You can pass that to np.poly1d(a)

polynominal=np.poly1d(a)

It returns the equation. I called it polynomial. But that equation will also do math for you. You give it an X and it gives you Y! So used linspace to make a bunch of x values! You want more than 2 now becuase your line will be curved.

x_fit=np.linspace(x.min(),x.max())

Now send your x_fit into polynominal and you have all your y's!

y_fit=polynominal(x_fit)

Now plot them!

I know that was all just weird. But that is python and object oriented code. It can do work for you.

If you are going to use textstr to make a string with the results of poly1d to show the equation it takes a trick or two. I did it two ways. First you could cast something to a string. So you could do

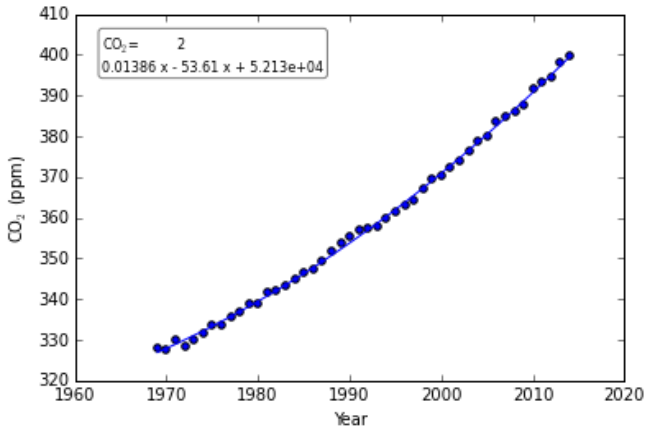
textstr='CO2={}'.format(poly1d(a))

Now for your y-axis If you want to subscript the 2 you can do the dollar sign trick for an equation with an underscore. I can't close the dollar signs as it will show the eqauation CO_2 so you would need to write CO_2

Double click on this box to see how I did the CO2

In [40]:
```
# Your code here!
```

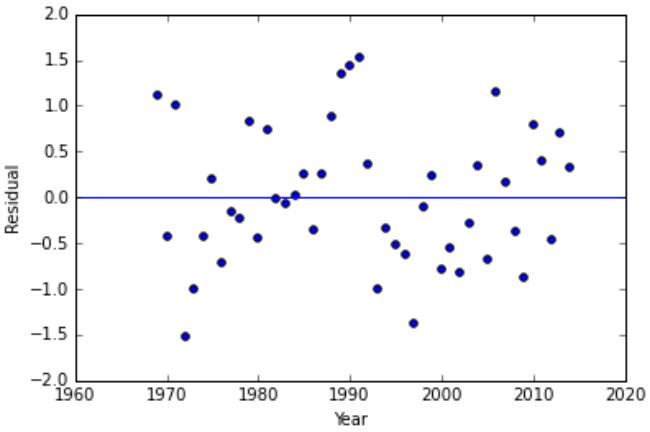
Out[40]:
```
<matplotlib.text.Text at 0x10870320>
```



Now that is a great fit!!!!!! We can also look at the residuals for it. Can you plot the residuals???

In [41]:
```
# Your code here!
```

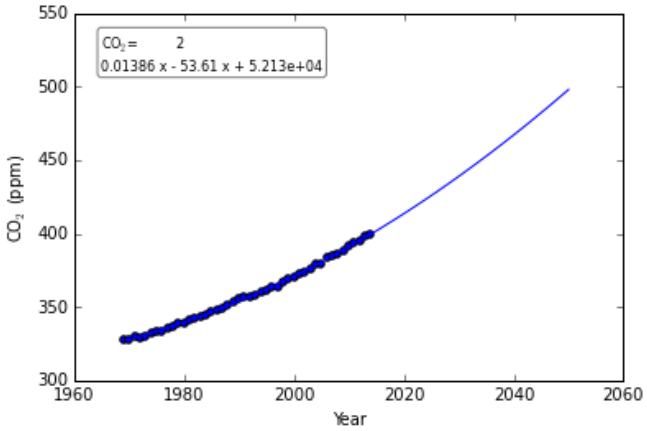
Out[41]:
```
<matplotlib.lines.Line2D at 0x1085f4a8>
```



Now make a prediction for CO2 for 2016-2050! Can you extend the line to 2050?

In [47]:
```
# Your code here
```

Out[47]:
```
<matplotlib.text.Text at 0x118087b8>
```

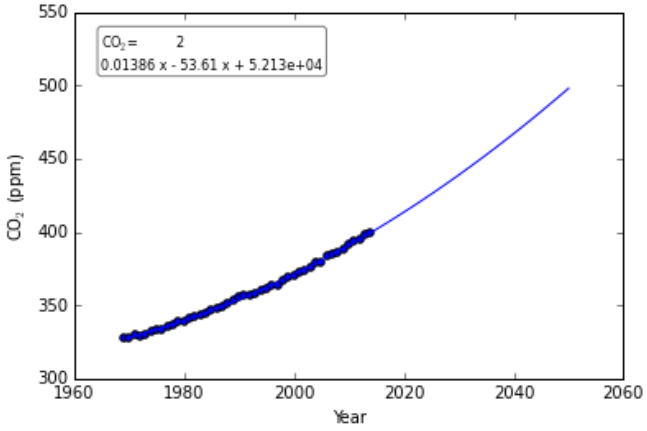


Comments!!!

I have been really bad about adding comments to the code I write. I try to write in markdown but you also want to add comments so people can follow the code. So lets take our last code and add comments to help us using the hastag! you will thank yourself in a few months from now. So comment your last code and I know you can do better than me!

In [54]:
```
#your code and comments here!
```

Out[54]:
```
<matplotlib.text.Text at 0x12282588>
```



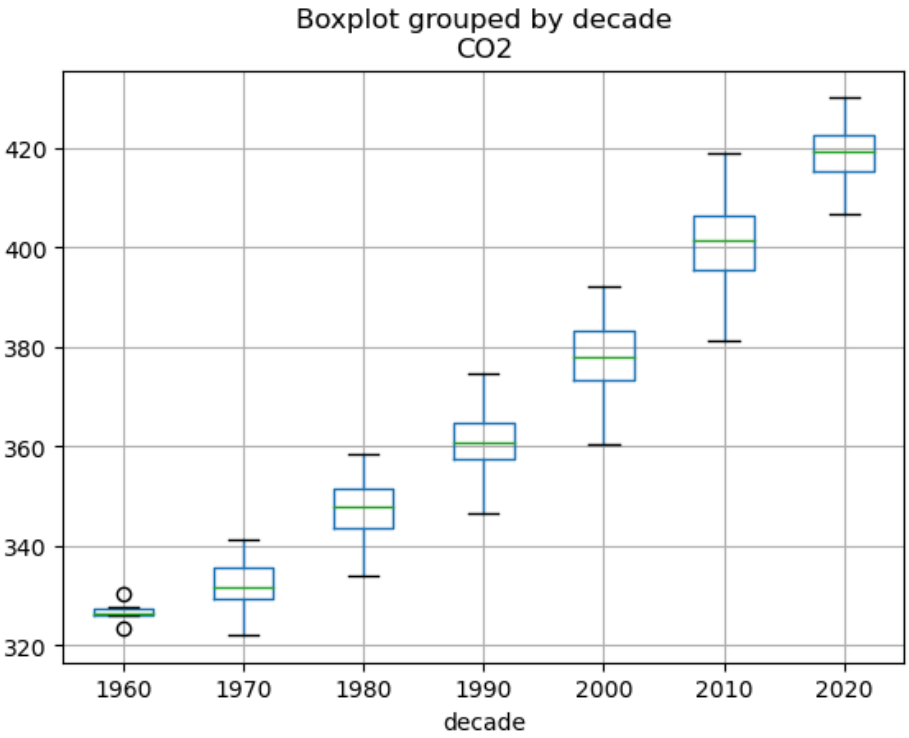
In [ ]:

In [ ]:

Answers

In [44]:
```
fig,ax=plt.subplots()
df_scripps.boxplot(column='CO2',by='decade',ax=ax)
```

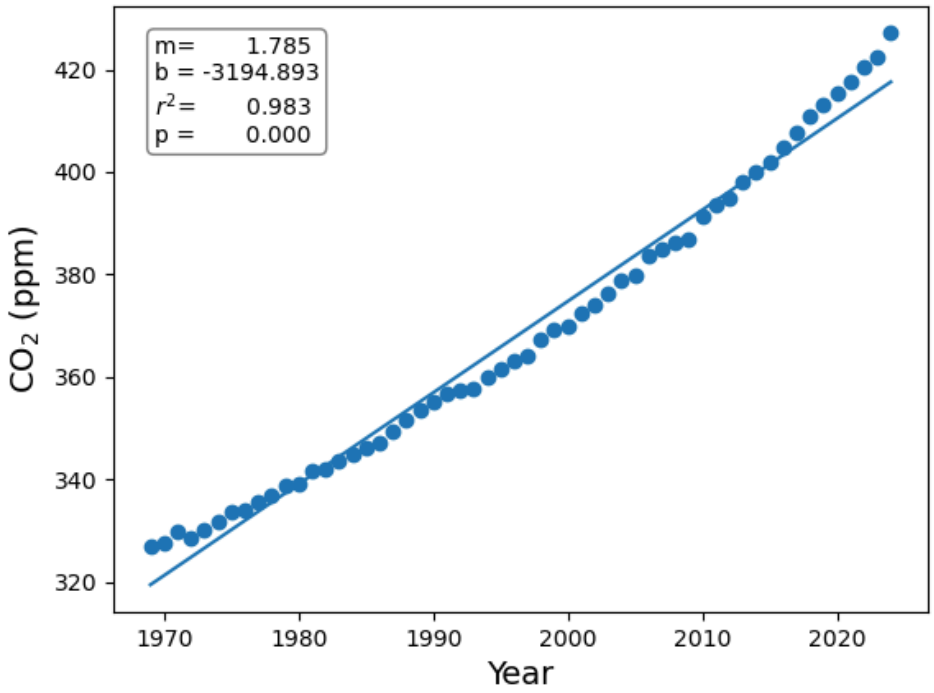
Out[44]:
```
<Axes: title={'center': 'CO2'}, xlabel='decade'>
```



In [114…
```
fig,ax=plt.subplots()
props=dict(boxstyle='round',facecolor='white',alpha=0.5)

x=dfS.CO2.resample('YE').mean().index.year
y=dfS.CO2.resample('YE').mean()
ax.scatter(x,y)

results=stats.linregress(x,y)
ax.plot(x,results[0]*x+results[1])
textstr='m={:>12.3f}\nb ={:>10.3f}\n$r^2$={:>12.3f}\np ={:>12.3f}'\
        .format(results[0],results[1],results[2]**2,results[3])
ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10
        ,verticalalignment='top',bbox=props)
ax.set_ylabel('CO$_2$ (ppm)',fontsize=14)
ax.set_xlabel('Year',fontsize=14)
```

Out[114…
```
Text(0.5, 0, 'Year')
```

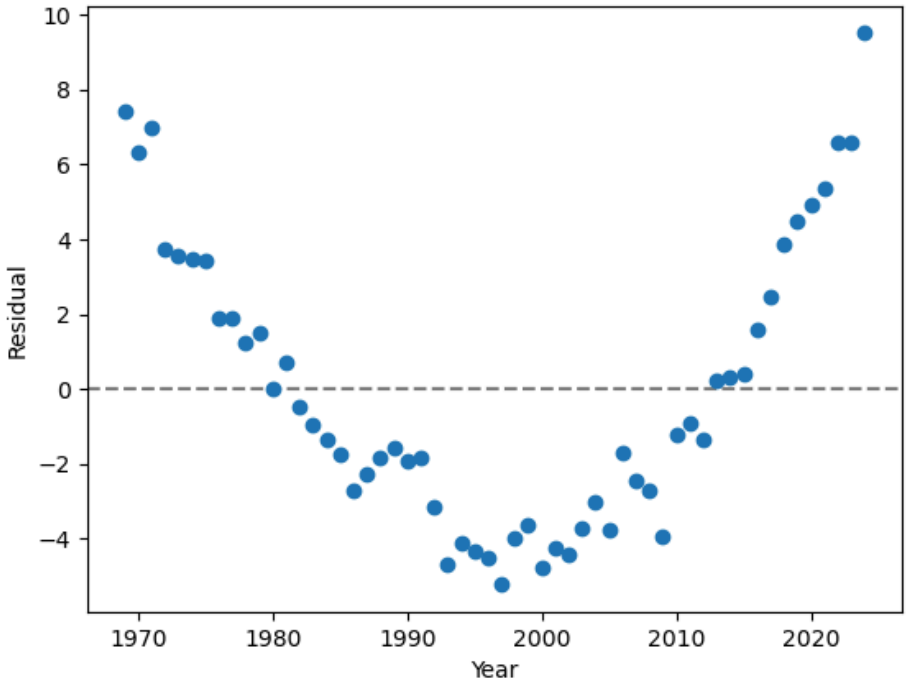
Figure



In [116…
```
fig,ax=plt.subplots()

x=dfS.CO2.resample('YE').mean().index.year
y=dfS.CO2.resample('YE').mean()

results=stats.linregress(x,y)
ax.scatter(x,y-(results[0]*x+results[1]))
ax.set_ylabel('Residual')
ax.set_xlabel('Year')
ax.axhline(y=0,color='grey',zorder=0,ls='--')
```

Out[116…
```
<matplotlib.lines.Line2D at 0x144713550>
```

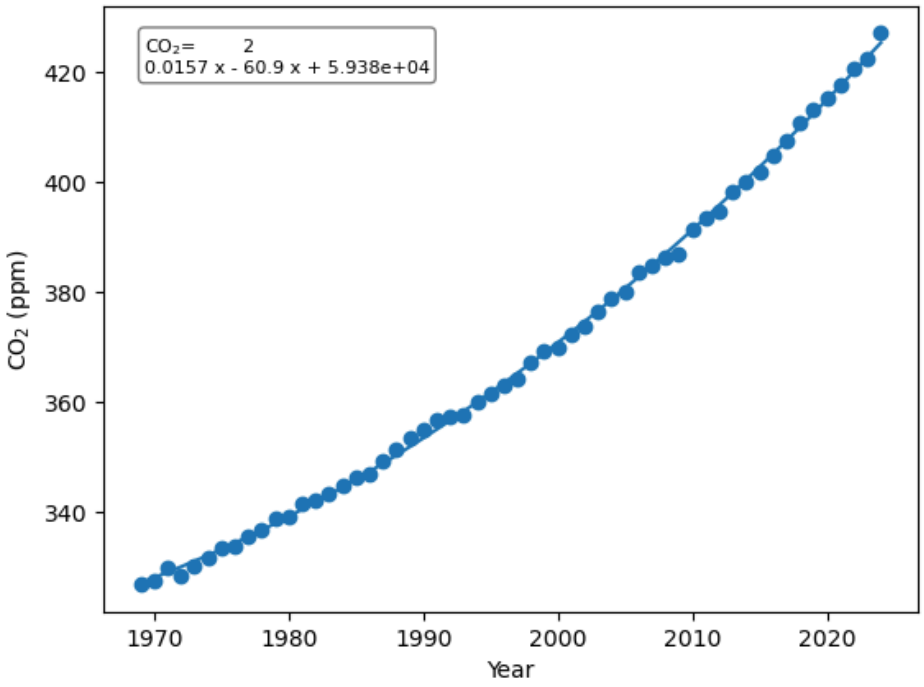
Figure



In [119…
```
fit_order=2

fig,ax=plt.subplots()
props=dict(boxstyle='round',facecolor='white',alpha=0.5)

x=dfS.CO2.resample('YE').mean().index.year
y=dfS.CO2.resample('YE').mean()

x_fit=np.linspace(x.min(),x.max())  #I set the range using the data
a=np.polyfit(x,y,fit_order)  #I just copied from above to have in the same cell
polynominal=np.poly1d(a)
y_fit=polynominal(x_fit)

textstr='CO$_2$={}'.format(polynominal)
ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=8
        ,verticalalignment='top',bbox=props)

ax.plot(x_fit,y_fit)
ax.scatter(x,y)
ax.set_ylabel('CO$_2$ (ppm)')
ax.set_xlabel('Year')
```

Out[119…
```
Text(0.5, 0, 'Year')
```

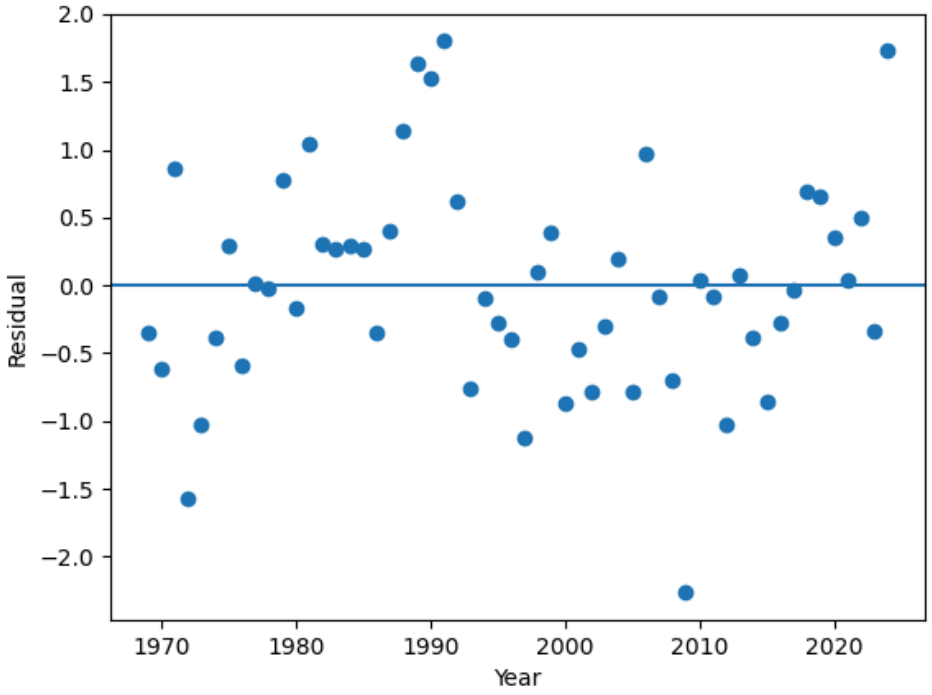
Figure



In [120…
```
fit_order=2

fig,ax=plt.subplots()

x=dfS.CO2.resample('YE').mean().index.year
y=dfS.CO2.resample('YE').mean()

x_fit=np.linspace(x.min(),x.max())
a=np.polyfit(x,y,fit_order)
polynominal=np.poly1d(a)

ax.scatter(x,y-polynominal(x))

ax.set_ylabel('Residual')
ax.set_xlabel('Year')
ax.axhline(y=0)
```

Out[120…
```
<matplotlib.lines.Line2D at 0x1447aac10>
```

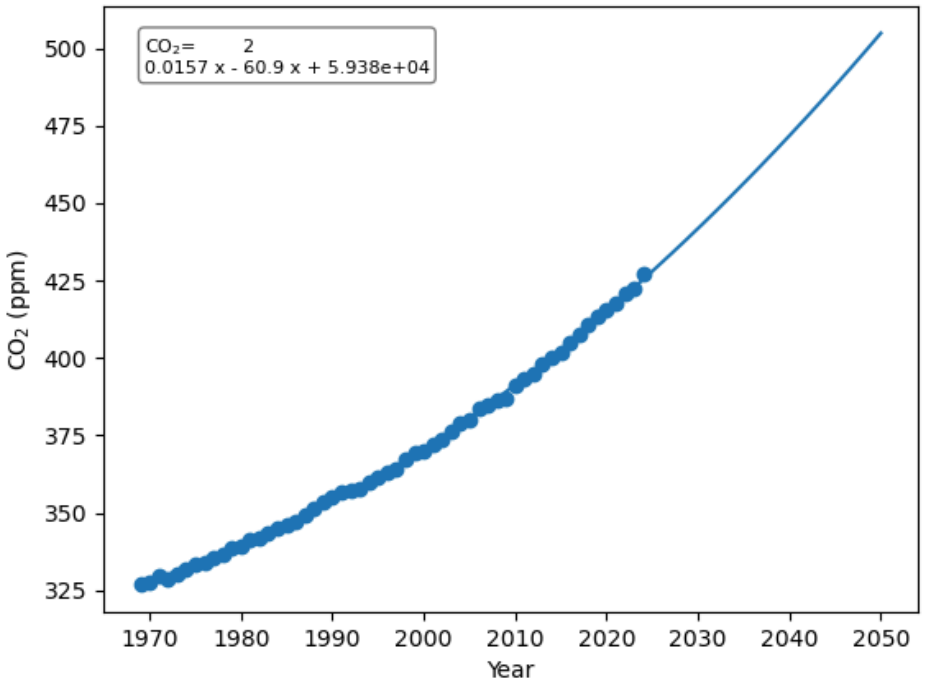
Figure



In [121…
```
fit_order=2

fig,ax=plt.subplots()
props=dict(boxstyle='round',facecolor='white',alpha=0.5)

x=df_scripps.CO2.resample('YE').mean().index.year
y=df_scripps.CO2.resample('YE').mean()

x_fit=np.linspace(1970,2050)
a=np.polyfit(x,y,fit_order)
polynominal=np.poly1d(a)
y_fit=polynominal(x_fit)

ax.plot(x_fit,y_fit)
ax.scatter(x,y)
ax.set_ylabel('CO$_2$ (ppm)')
ax.set_xlabel('Year')

textstr='CO$_2$={}'.format(np.poly1d(a))
ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=8
        ,verticalalignment='top',bbox=props)
```

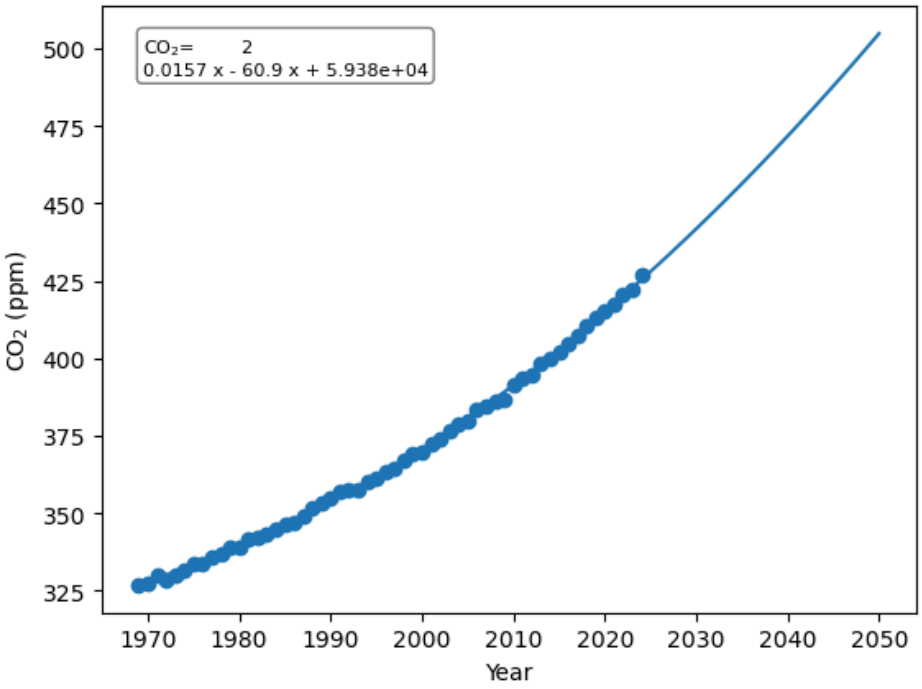
Out[121…
```
Text(0.05, 0.95, 'CO$_2$=        2\n0.0157 x - 60.9 x + 5.938e+04')
```

Figure



In [50]:
```
#get the x and y data from the larger data set for plotting and statistics.
x=df_scripps.CO2.resample('YE').mean().index.year
y=df_scripps.CO2.resample('YE').mean()

#i am going to fit the data using poly1d and use linspace to give us a bunch of points.
fit_order=2   #this is the order of the line.
x_fit=np.linspace(1970,2050)
a=np.polyfit(x,y,fit_order)
polynominal=np.poly1d(a)
y_fit=polynominal(x_fit)

#plot the data, the fit, label the axes and add the equation using a text box.
fig,ax=plt.subplots()
ax.plot(x_fit,y_fit)
ax.scatter(x,y)
ax.set_ylabel('CO$_2$ (ppm)')
ax.set_xlabel('Year')

props=dict(boxstyle='round',facecolor='white',alpha=0.5)
textstr='CO$_2$={}'.format(np.poly1d(a))
ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=8
        ,verticalalignment='top',bbox=props)
```

Out[50]:
```
Text(0.05, 0.95, 'CO$_2$=        2\n0.0157 x - 60.9 x + 5.938e+04')
```



In [ ]:

In [1]:
```
ls
```

```
2025_daily_merge_co2_ljo.csv         excel-custom.png
2025_daily_merge_co2_ljo_fixed.csv   excel-date.png
MLO-Bonus-BoxModel-Answers.ipynb     excel_header.jpg
MLO-Bonus-BoxModel.ipynb             excel_header.png
MLO-Bonus-BoxModel.pdf               groupby_since_2000.jpg
WaterBucketImage.jpg                 pandas-mauna-lao.ipynb
bucket.jpeg                          pandastimeseries.ipynb
daily_merge_co2_ljo.csv              pandastimeseries.pdf
excel-choose.png                     rolling.jpg
```

In [ ]: